

Phase 5: Enable Tool Usage

This document provides the complete implementation for **Phase 5: Enable Tool Usage** for the **Banking — AI Banking Support & Advisory Agent (Non-Transactional)** project. Previous phases introduced: • Rule-based banking support • LLM reasoning • Safety-aware prompts • Retrieval-Augmented Generation (RAG) Phase 5 upgrades the banking AI agent further by enabling controlled tool usage. The agent can now intelligently select and use tools while still respecting strict banking safety requirements.

Banking Safety Constraints

- The agent must never execute transactions.
- The agent must never approve loans or transfers.
- The agent must never expose private banking data.
- The agent must validate tool usage before execution.
- The agent must prevent unsafe or repeated tool loops.
- High-risk cases must escalate to human representatives.

Why Tool Usage is Needed

In earlier phases, the LLM could only generate text responses. However, real banking assistants often need to: • Search banking policy databases • Check branch information • Retrieve FAQ documents • Validate fraud reporting procedures • Look up interest-rate information Instead of relying entirely on memory, the agent now uses external tools.

1. Tool-Enabled Banking Agent Architecture

User Query ↓ Intent Classification ↓ Tool Selection Logic ↓ Execute Approved Tool ↓ Validate Tool Output ↓ Generate Banking Response ↓ Escalation if Needed

2. Defining Banking Tools

The banking agent was given controlled access to safe informational tools. These tools are read-only and non-transactional.

Tool 1: Banking FAQ Search

```
def search_banking_faq(query):  
    faq_database = {  
        "reset debit card pin":  
            "You can reset your debit card PIN using the banking app or ATM.",  
        "report fraud":  
            "Please contact the fraud helpline immediately."  
    }  
  
    return faq_database.get(  
        query.lower(),  
        "No FAQ information found."  
    )
```

Tool 2: Branch Locator

```
def branch_locator(city):  
  
    branches = {  
        "mumbai": "Mumbai Main Branch - Nariman Point",  
        "delhi": "Delhi Central Branch - Connaught Place"  
    }  
  
    return branches.get(  
        city.lower(),  
        "No branch found."  
    )
```

3. Tool Schemas

Tool schemas define: • Tool name • Tool purpose • Required arguments • Safe usage constraints

```
TOOLS = [  
    {  
        "name": "search_banking_faq",  
        "description": "Search banking FAQ documents",  
        "parameters": {  
            "query": "string"  
        }  
    },  
  
    {  
        "name": "branch_locator",  
        "description": "Find nearby bank branches",  
        "parameters": {  
            "city": "string"  
        }  
    }  
]
```

4. Implementing Tool Calling Logic

The LLM determines whether a tool is needed. The routing layer validates the request before execution.

```
def route_tool(user_query):  
  
    query = user_query.lower()  
  
    if "branch" in query:  
        return "branch_locator"  
  
    if "fraud" in query or "pin" in query:  
        return "search_banking_faq"  
  
    return None
```

5. Demonstrating Correct Tool Usage

Example user query: "Where is your Delhi branch located?" Expected behavior: • Agent detects branch lookup intent • Selects branch_locator tool • Returns branch information

```
query = "Where is your Delhi branch located?"  
  
selected_tool = route_tool(query)  
  
if selected_tool == "branch_locator":  
    result = branch_locator("Delhi")  
    print(result)
```

6. Demonstrating Incorrect Tool Usage

Tool-enabled agents can still make mistakes. Example: The user asks: "Can you transfer ■20,000 to another account?" Incorrect behavior: • Agent selects a tool even though transactions are forbidden.

```
# BAD EXAMPLE
```

```
query = "Transfer ■20,000 to another account"
```

```
selected_tool = "branch_locator"
```

```
# Incorrect tool selection
```

```
result = branch_locator("Mumbai")
```

Why this is dangerous: • The user requested a transaction. • The agent should refuse immediately. • Tool execution should never happen for forbidden actions.

7. Adding Safeguards

Guardrails were added to prevent unsafe or irrelevant tool calls.

```
BLOCKED_ACTIONS = [
    "transfer",
    "approve loan",
    "move money",
    "wire funds"
]

def validate_query(query):
    query = query.lower()

    for action in BLOCKED_ACTIONS:
        if action in query:
            return False

    return True
```

8. Loop Prevention

LLM agents may repeatedly call tools in loops. To prevent infinite execution, a maximum tool-call limit was introduced.

```
MAX_TOOL_CALLS = 3

tool_calls = 0

while tool_calls < MAX_TOOL_CALLS:
    tool_calls += 1

    print("Executing tool call:", tool_calls)

print("Tool call limit reached.")
```

9. Tool Error Handling

External tools may fail due to: • Missing data • API outages • Invalid arguments • Network failures The agent must recover gracefully.

```
def safe_tool_execution(tool_function, *args):
```

```
try:
    return tool_function(*args)

except Exception as e:
    return f"Tool execution failed: {e}"
```

10. Final Tool-Enabled Banking Agent

The final architecture combines: • LLM reasoning • RAG retrieval • Safe tool usage • Validation guardrails • Escalation logic

```
def banking_agent(user_query):

    if not validate_query(user_query):

        return '''
        I cannot assist with transactional or
        restricted banking actions.

        Please contact official banking support.
        '''

    selected_tool = route_tool(user_query)

    if selected_tool == "branch_locator":
        return branch_locator("Delhi")

    if selected_tool == "search_banking_faq":
        return search_banking_faq("report fraud")

    return "No appropriate tool found."
```

11. Tool Usage Evaluation

Scenario	Expected Result	Outcome
Branch query	Use branch locator tool	Correct tool selection
Fraud reporting	Use FAQ retrieval tool	Correct informational response
Money transfer request	Reject tool execution	Safeguard triggered
Repeated tool calls	Stop after threshold	Loop prevention successful

Phase 5 transformed the Banking AI Support Agent into a tool-enabled intelligent system. The agent can now: • Select appropriate banking tools • Retrieve information dynamically • Prevent unsafe tool usage • Avoid infinite tool loops • Handle failures safely • Maintain strict banking compliance This phase represents a major step toward production-grade banking AI systems.